

String Hashing

Lecture 9

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Motivation & Real-World Context
2. Polynomial Rolling Hash
3. The Rabin-Karp Algorithm
4. Collision Probability & The Birthday Paradox
5. Multiset (Commutative) Hashing
6. XOR Hashing

Motivation & Real-World Context

The Cost of Naïve Comparison

- Comparing two strings of length N takes $\mathcal{O}(N)$ time.
- **Question:** How do systems perform massive-scale text operations efficiently?

Examples:

- **Plagiarism checkers** scanning millions of documents.
- **IDEs** finding all variable usages instantly — acting as an *inverted index* that converts text into $\mathcal{O}(1)$ integer lookups.

Hashing in the Real World

Content-Addressable Storage Git uses hashes (SHA-1) to uniquely identify commits, trees, and blobs.

Distributed Systems Consistent hashing dictates data partitioning and load balancing in cloud architectures.

Security vs. Speed Cryptographic hashes (SHA-256) are intentionally *slow* and one-way. Algorithmic / rolling hashes are optimised for blazing-fast data-structure operations.

Polynomial Rolling Hash

Express a string similarly to a base- p number system:

$$H(S) = \sum_{i=0}^{|S|-1} S[i] \cdot p^i \pmod{m}$$

p (**Base**) A prime $\approx$ alphabet size (e.g. 31 for lowercase English).

m (**Modulo**) A large prime (e.g. $10^9 + 7$) to prevent integer overflow.

$\mathcal{O}(N)$ Precomputation

Compute the hash of all prefixes in $\mathcal{O}(N)$:

```
1  const int p = 31;
2  const int m = 1e9 + 7;
3  vector<long long> p_pow(s.size());
4  vector<long long> h(s.size() + 1, 0);
5
6  p_pow[0] = 1;
7  for (int i = 1; i < (int)p_pow.size(); i++) {
8      p_pow[i] = (p_pow[i-1] * p) % m;
9  }
10 for (int i = 0; i < (int)s.size(); i++) {
11     h[i+1] = (h[i] + (s[i] - 'a' + 1) * p_pow[i]) % m;
12 }
```


$\mathcal{O}(1)$ Substring Extraction

Extract the hash of any substring $S[i \dots j]$:

$$H(S[i \dots j]) = (H[j+1] - H[i]) \cdot p^{-i} \pmod{m}$$

Avoiding the modular inverse:

- Multiply the smaller hash by p^{j-i+1} to align polynomial powers before comparing — no p^{-i} required.

The Rabin-Karp Algorithm

Fast Pattern Matching

Find a pattern of length M within a text of length N :

1. Compute the hash of the target pattern.
2. Compute the rolling hash of every window of length M .
3. On a hash match, verify character-by-character.

Complexity: $\mathcal{O}(N + M)$ average time.

Collision Probability & The Birthday Paradox

The Birthday Paradox

With $m = 10^9 + 7$ you do *not* need 10^9 strings for a collision.

The Math N strings create $\frac{N(N-1)}{2}$ pairs — collisions grow quadratically.

The Threshold 50 % collision probability at $\approx \sqrt{m} \approx 31,600$ strings.

Conclusion A single modulo is statistically unsafe for large datasets.

Hash the string *twice* with two distinct large moduli:

$$m_1 = 10^9 + 7, \quad m_2 = 10^9 + 9$$

- Effective space expands to $\approx 10^{18}$.
- A collision now requires $\sqrt{10^{18}} = 10^9$ strings — practical certainty for all real-world scenarios.

Multiset (Commutative) Hashing

Polynomial hashing is order-dependent: "abc" $\neq$ "cba".

What if order *does not matter*?

Technique:

1. Assign a random 64-bit integer to every possible element.
2. Hash of the multiset = $sum \pmod{M}$ of those integers. Addition is commutative, so order vanishes.

Problem: Range Anagram Queries

Given an array of N characters, answer Q queries: “Do chars in $[l_1, r_1]$ form an anagram of chars in $[l_2, r_2]$?”

Standard Frequency arrays cost $\mathcal{O}(26)$ per query.

Hashing Assign random 64-bit hashes to 'a'–'z', build a prefix-sum array, then:

$$\text{Hash}(l_1, r_1) = \text{Hash}(l_2, r_2) \quad \text{in } \mathcal{O}(1).$$

XOR Hashing

The Nilpotent Property

Track a **dynamic state** with frequent additions and removals. XOR ($\oplus$) has two key properties:

Commutativity $A \oplus B = B \oplus A$

Nilpotence $A \oplus A = 0$

XOR Hashing:

- Assign a random 64-bit integer to every state element.
- XOR together to form the state hash.
- To *remove* an element, XOR its value again.

Problem: Dynamic Set Equality

Given two empty sets A and B , process Q add/remove queries and after each check $A \stackrel{?}{=} B$.

Solution:

- Maintain one 64-bit integer per set.
- Add/remove X from A : $\text{hashA} \hat{=} \text{random_map}[X]$
- Equality check: $\text{hashA} == \text{hashB}$ in $\mathcal{O}(1)$.

Real-world use: Chess engines use XOR hashing to track board states and detect three-fold repetition draws.

Questions?



cp-algorithms.

<https://cp-algorithms.com/string/string-hashing.html>



String Hashing.

<https://www.youtube.com/watch?v=hxcbkuqQeZQ&t=4891s>



XOR Hashing Tutorial.

<https://codeforces.com/blog/entry/85900>



Birthday Paradox.

https://en.wikipedia.org/wiki/Birthday_problem